AI Technical Assignment - Retail Company

The Challenge: Design and Build a Data Analysis Chat Assistant

You are building an internal data agent for a Retail Company's non-technical executive team (Store and Regional Managers). These managers need to ask complex questions about sales, inventory, and performance (e.g., "Why is the X branch underperforming? And how does it compare to our branch in Y?")

The agent will have access to:

- The Database: A read-only connection to a SQL database containing raw transaction logs.
- The "Golden Knowledge" Bucket: A data lake containing historical "Trios" (Question → SQL Query → Analyst Report) created by human experts.

The Task: Design a **production**-ready full High-Level Design (HLD), accompanied by a **detailed** technical explanation, and build a chat-agent prototype that allows executives to query this data naturally and discuss about it. The system should be easily extendable for new capabilities (generating graphs, sending reports via mail, etc) and data sources.

Note: We expect to see what services you aim to use, what the communication is between the components, how and where data is stored and handled. The "detailed technical explanation" needs to be detailed enough for us to understand how this system will function in **production.**

Requirements:

1. **Hybrid Intelligence:** The agent cannot rely on SQL alone. It must use the "Golden Bucket" to understand *how* analysts previously interpreted questions and apply similar logic to new queries. Explain how you will update the golden bucket over time and how you will provide relevant data at query time.
2. **Safety & PII Masking:** The agent is only allowed to answer analysis questions and needs to be safeguarded against malicious users. The raw transaction logs contain Customer Phones and Emails. The agent is **strictly forbidden** from displaying this PII in the final output, even if the SQL query retrieves it.
3. **High-Stakes Oversight (Destructive Ops):** While the DB is read-only, the agent manages a "Saved Reports" library. The agent must support inputs like *"Delete all reports mentioning Client X" / "Delete all the reports we made today"* This is a destructive action and requires a strict confirmation flow before execution, without breaking UX - the users are allowed to delete their own reports.
4. **Continuous Improvement (The Learning Loop):**
    a. **User Level:** The agent should remember that "Manager A" prefers tables while "Manager B" prefers bullet points.
    b. **System Level:** The agent should be able to learn from past interactions.

5. **Resilience & Graceful Error Handling:** The system must detect SQL syntax errors or empty returns and attempt to **self-correct** before giving up, without crashing the user interface and without inflating costs. The system should be resilient to API/3rd party services failures/downtime
6. **Quality Assurance:** How do you evaluate the agent before deployment? How do you verify that the generated reports answer users intent correctly?
7. **Observability:** We need to know when the agent is failing and why. Define the metrics you would track at the agent level and how you would support deep dive analysis/debugging (understanding what the message correspondence is and what went wrong).
8. **Agility (Persona Management):** The CEO wants to change the "tone" of the reports weekly. The system must allow non-developers to update the agent's instructions without redeployment.

Deliverables:

1. **Architecture Diagram (Mermaid etc):** Highlighting the building blocks, services and flow of the system based on the requirements above. In case you are planning to use a framework/service/data store for one of the building blocks, specify which one and include an explanation.
2. **Detailed technical explanation covering:**
    a. Reasoning for the chosen Cloud services / LLM models/frameworks used.
    b. Data flow between components (if needs to elaborate on HLD).
    c. Error handling and fallback strategies.
    d. Setup Instructions and example run
    e. Make sure to include a detailed explanation of how you handle/solve each of the requirements
3. **Working Code/Prototype:** Build a **chat agent** that can generate and run SQL queries and self heal if needed against the DB listed below and create a report. The prototype needs to support at least 2 of the following requirements (defined above):
    a. Safety & PII Masking
    b. High-Stakes Oversight
    c. Resilience & Graceful Error Handling
    d. Quality Assurance
    e. Observability
4. **Simple** CLI-based interface for chat interactions. (UI's won't gain any additional points)
5. **Your solution must be runnable on another machine (Docker is not a must, just proper setup instructions).**
6. Use a framework of your choice (preferably **Lang Graph / Lang Chain V1**).

Note: the prototype needs to be simple and working.

Our assessment will focus mainly on system design, the technical explanation, and an elegant Prototype

Dataset Specification:

- **Dataset:** `bigquery-public-data.thelook_ecommerce`
- **Required Tables:**

- `orders` - Customer order information
- `order_items` - Individual items within orders
- `products` - Product catalog and details
- `users` - Customer demographics and information

Expected Agent Capabilities:

Your prototype agent should be able to perform data analysis and generate insights such as:

- Customer behavior (e.g., top customers, total spend)
- Product performance
- Time-based metrics (e.g., monthly revenue, up-to-date revenue by product)
- Answer questions about the general structure of the database

1. **Use BigQuery integration** to query and analyze the specified tables. Your agent should be able to construct and execute SQL queries dynamically based on the analysis requirements.
2. You should preferably use one of the newer **Google Gemini models**. You can get a free API key from Google AI Studio. Please be mindful of the rate limits. Alternatively, you can use OpenRouter or Ollama if you prefer (or have issues with rate limits) . We have created a simple client for your convenience: https://github.com/Opsfleet/lc-openrouter-ollama-client

Setup Instructions

Environment Setup

1. **Install Python dependencies:**

```
pip install -r requirements.txt
```

GCP/BigQuery Setup

1. **Set up BigQuery access** by following the BigQuery Client Libraries documentation if you don't already have BigQuery access configured.
2. **Free Tier:** Google Cloud provides 1TB of free BigQuery compute per month, which is more than sufficient for this challenge.
3. **Authentication:** Ensure your environment is authenticated with Google Cloud to access the public datasets.

Time Expectation

We expect this assignment to take between 6 to 12 hours of work.

Submission

Share with us a **your public GitHub repository** with:

- Documentation
- Source code
- Architecture diagram

Resources

Quick Starts

- https://docs.langchain.com/oss/python/langchain/quickstart
- https://docs.langchain.com/oss/python/langgraph/quickstart
- https://docs.langchain.com/oss/python/langgraph/add-memory

bq_client.py
Alex Freidman 11 Aug 2025

```python
import logging
from typing import Optional, List, Dict, Any
import pandas as pd
from google.cloud import bigquery
```

requirements.txt

Alex Freidman 11 Aug 2025

```
langgraph>=0.2.0
langchain-google-genai>=1.0.0
google-cloud-bigquery>=3.13.0
pandas>=2.0.0
python-dotenv>=1.0.0
```